



Vulkan Compute Support in DCompute and LDC

Google Summer of Code 2026

Presented by Pham Quang Ha.



About Me & GSoC Project

About Me

Pham Quang Ha

- An IT student at Phenikaa University in Vietnam.
- Passionate about learning new technologies.

Finding Dlang

- I discovered Dlang through Google Summer of Code (GSoC).
- It looked like a very unique and interesting programming language, so I decided to study it deeply.

Why This Project?

- Other popular projects had too many applicants.
- Vulkan backend is a hard topic, and I want to challenge my programming skills.



Why Target Vulkan for D?

- Cross-platform GPU compute capability via SPIR-V.
- Avoids proprietary vendor lock-in.
- **Goal:** Write GPU compute kernels entirely in pure D and run them natively on Vulkan hardware.

The Compilation Architecture

D Source Code

Written in D with
DCompute annotations
(@compute, @kernel).

LDC

Compiles D code into
Vulkan-ready LLVM IR.

SPIR-V & Vulkan

LLVM SPIR-V:
Generates .spv bytecode.
Vulkan GPU: Executes the
generated .spv file.

Writing the Kernel in Pure D

Use the **@compute** attribute to designate GPU kernels natively:

```
@compute(CompileFor.deviceOnly) module vulkan_minimal_kernel;
import ldc.dcompute;

@kernel() void minimal_kernel(GlobalPointer!float output) {
    *output = 67.36f;
}
```

Compiling and Execution

```
ldc2 -c -O3 -mdcompute-targets=vulkan-130 test.d
```

Compile

Run the command above to generate the `test.spv` file.

Execute

Load `test.spv` into a runner like `vulkan_minimal_compute` to execute on the GPU and retrieve results.

Demo Video





The Beginner's Journey: Facing Challenges

01

Massive Codebase

Navigating over
160,000 lines of C++
code in LDC.

02

Complex Build

Linker errors and
static/dynamic
mismatches on LLVM
23/24.

03

ABI & Upstream

Fixing ABI tests and
dealing with
unexpected LLVM
issues.

04

Language Barrier

Overcoming the fear of
asking questions in
English.

Challenge 1: The Massive Codebase

Hundreds of thousands of lines of C++ code.

01

Strategy

Start small with easy-labeled issues in DMD before building LDC.

02

Support

Mentor guidance (narrowing scope, print debugging).

03

Mindset

"Learn by doing" with basic AI Q&A, avoiding over-reliance.

Challenge 2: The Complex Build

Linker errors and static/dynamic mismatches on LLVM 23/24.

01

Bleeding-Edge

Targeting bleeding-edge LLVM (versions 23 and 24).

02

Linking Mismatch

Static vs. Dynamic linking mismatches.

03

Linker Errors

Endless linker errors before even writing code.

Challenge 3: ABI Tests & The LLVM Incident

Fixing ABI tests, upstream challenges, and cross-repository debugging.

01

ABI Tests

Fixing ABI tests required deep changes in LLVM.

02

The Incident

Accidentally pushed a Vietnamese comment into the LLVM upstream!

03

Result

Had to debug and fix code in both LDC and LLVM repositories.

■ The LLVM Incident

"I accidentally pushed a Vietnamese comment into the LLVM upstream!"

FinalEvolution on D Forum

"Ps. You left a little Vietnamese in llvm/lib/Target/SPIRV/
SPIRVLegalizePointerCast.cpp:233"



Nicholas Wilson 18/08/2026 12:49 CH

you still have some comments in Vietnamese there

Challenge 4: The Language Barrier

Overcoming communication anxiety and finding support in open source.

01

Non-Native English

- Not a native speaker.
- Technical communication felt daunting.

02

Anxiety & Hesitation

- Fear of asking "stupid" questions.
- Worried about being misunderstood.

03

Result / Lesson

- The community is incredibly supportive.
- Language is never a barrier.

Lessons Learned & Advice

Personal takeaways and key recommendations for future open-source contributors.

RETROSPECTIVE

What I Learned

- Patience is key when reading massive codebases.
- Communication is just as important as writing code.

RECOMMENDATIONS

My Advice

- Start small (look for "good first issues").
- Don't be afraid to ask questions or make mistakes.
- Use AI to learn, not to replace your own thinking.

Current Status & Future Roadmap

Done

Vulkan in LDC & LLVM

- Added Vulkan 1.3 support in LDC.
- Fixed pointer and buffer bugs in LLVM.

Done

Tested on Real GPU

- Ran compute shaders on real GPU.
- Minimal kernel and Vector Add work correctly.

Next Steps

Upstream Pull Requests

- Merge PR #5132 into LDC master.
- Merge PR #216919 into LLVM main.

Future (SAOC Plan)

Auto Testing & `tensord`

- Integrate automated GPU tests into `lit`.
- Build `tensord` library (Matrix Multiply, ReLU, Sigmoid).



Thank you for listening!

Acknowledgments

Special thanks to the **D Language Foundation** community and **Google Summer of Code**.

Immense gratitude to my mentor, **Nicholas Wilson**, for his guidance and patience throughout this journey.

Questions?

Feel free to ask any questions!

 github.com/gnavdev28

 ha.phamquang.dev@gmail.com